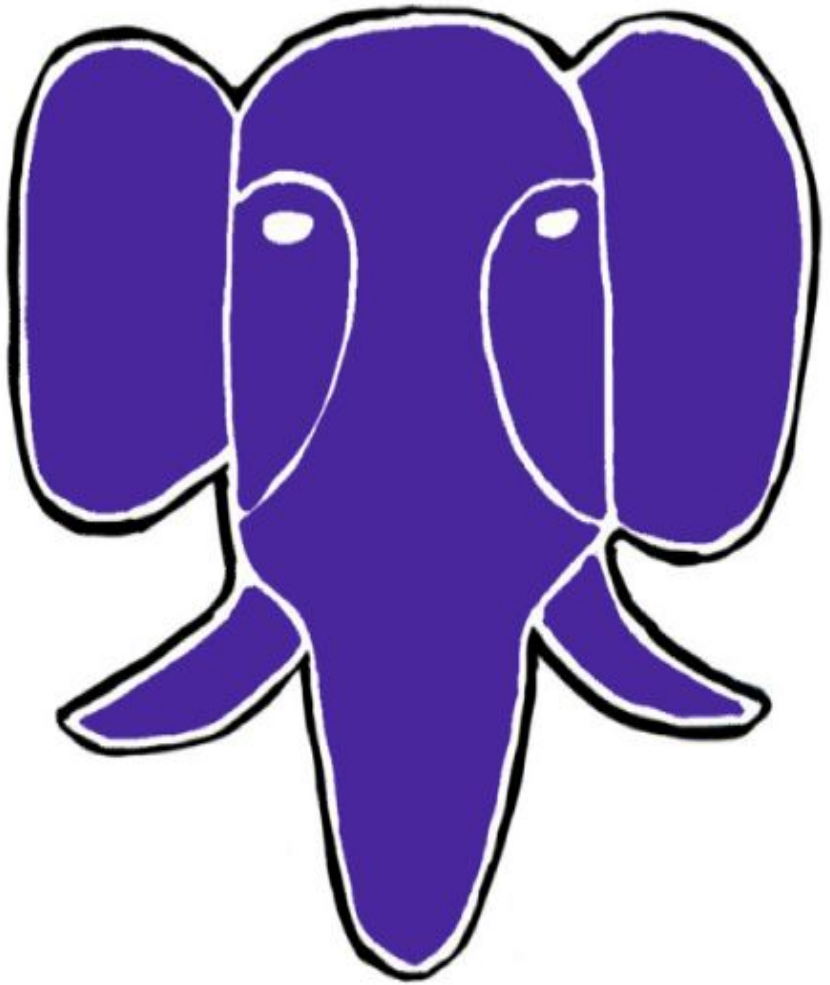
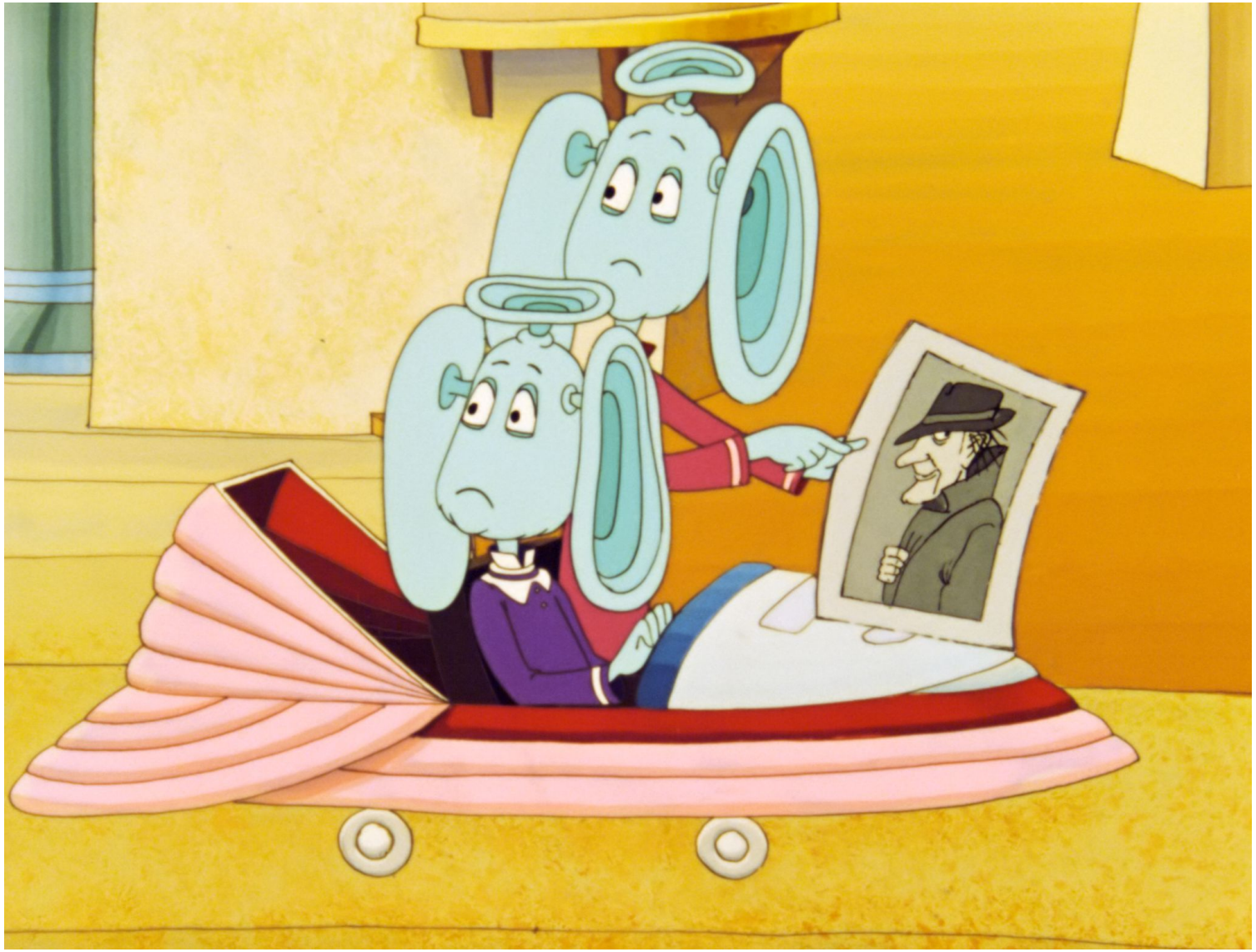
Евгений Аристов

Открытый урок replication in PostgreSQL



https://aristov.tech



https://aristov.tech

https://aristov.tech

Аристов Евгений Николаевич



https://aristov.tech

Founder & CEO aristov.tech
25 лет занимаюсь разработкой БД и ПО

Архитектор высоконагруженных баз данных и инфраструктуры

Спроектировал и разработал более ста проектов для финансового сектора, сетевых магазинов, фитнес-центров, отелей.

Сейчас решаю актуальные для бизнеса задачи: аудит и оптимизация БД и инфраструктуры, миграция на PostgreSQL, обучение сотрудников.

Автор более 30 практических курсов по PostgreSQL, MySQL, Mongo и др..

Автор книг по PostgreSQL. Новинка [PostgreSQL 18: для архитекторов систем]

https://aristov.tech

Правила вебинара

Задаем вопрос в чат

Вопросы вижу, отвечу в момент логической паузы

Если есть вопрос голосом - поставьте знак ? в чат

Если остались вопросы, можно написать мне через сайт aristov.tech

https://aristov.tech

Маршрут вебинара

https://aristov.tech

Многодетный Postgres - от асинхронного хаоса до каскадного воспитания.

✅**У нас будет пополнение:**

- как бизнес приходит к мысли, что одного родителя уже мало
- рождение первой реплики: радость, которая быстро сменяется бессонными ночами

✅**Кризис трех лет:**

- почему реплики узнают новости позже родителей
- пошла теплая (реплика)

✅**Каскадное воспитание:**

- если реплик стало много и старшие следят за младшими
- плюсы и минусы «дедовщины» в кластере

✅**«Детские пособия», которые разоряют семью**:

- что гарантируют слоты репликации
- реплика «ушла в армию»

✅**Когда они вырастают:**

https://aristov.tech

- папа «прилег отдохнуть»
- Patroni - нотариус для делёжки наследства

https://aristov.tech

У нас будет пополнение

https://aristov.tech

Как бизнес приходит к мысли, что одного родителя уже мало

https://aristov.tech

Бэкап это хорошо, но нужно время на его развертывание.

Чтобы обеспечить высокую доступность (HA) и придумали **репликацию**.

Кроме этого:

- Что делать, когда закончились физические ядра и память у сервера? горизонтально масштабировать
- Бэкап лучше делать с реплики, а не мастера.
- Георасределение нагрузки.
- Нагрузку по чтению и отчетам можно переложить на реплику

https://aristov.tech

Рождение первой реплики: радость, которая быстро сменяется бессонными ночами

https://aristov.tech

Какую выбрать???

Варианты со своими плюсами и минусами:

- физическая
  - pg_basebackup
- логическая
  - pub/sub

https://aristov.tech

https://aristov.tech

Физическая репликация

Особенности:

- мастер-слейв: поток данных только в одну сторону
- трансляция потока журнальных записей - WAL
- *требуется двоичная совместимость серверов - одна и та же версия*
- возможна репликация только всего кластера
- pg_basebackup с ключом -R - прописывает все параметры и обеспечивает постоянное соединение
- wal_keep_size + слот

https://aristov.tech

https://aristov.tech

Физическая репликация

Команды, которые допускаются на реплике:

- создание резервной копии (pg_basebackup)
- установка параметров сервера (set, reset)
- запросы на чтение данных (select, copy to, курсоры)
- управление транзакциями (begin, commit, rollback...)

Не допускаются на реплике:

- команды DDL (create, drop...), в том числе создание временных таблиц
- любые изменения (insert, update, delete, truncate, nextval...)
- блокировки, предполагающие изменение (select for update...)
- команды сопровождения (vacuum, analyze, reindex...)
- управление доступом (grant, revoke...)
- не срабатывают триггеры и пользовательские (advisory) блокировки

Что происходит с Vacuum:

- зависит от параметра hot_standby_feedback

https://aristov.tech

https://aristov.tech

Логическая репликация

https://aristov.tech

https://aristov.tech

Логическая репликация

Особенности:

- *поставщик-подписчик: поток данных возможен в обе стороны*
- возможна выборочная репликация отдельных таблиц
- используется информация о строках (необходимо повысить уровень WAL до logical)
- *возможна репликация между разными основными версиями PostgreSQL*

Кейсы использования:

- кастомная логика
- миграция на новую версию Постгреса
- объединение данных из разных источников
- репликация отдельных таблиц, когда весь сервер не нужен
- рассылка справочников по территориальным подразделениям

https://aristov.tech

https://aristov.tech

Логическая репликация. Публикующий сервер

- выдает изменения данных построчно в порядке их фиксации
- параметр wal_level = logical
- реплицируются команды INSERT, UPDATE, DELETE, в 11 версии добавили TRUNCATE
- **DDL не передаются**, то есть таблицы-приемники на стороне подписчика надо создавать вручную
- возможна начальная синхронизация
- используется слот логической репликации
- реплицировать данные возможно только из базовых таблиц в базовые таблицы. То есть таблицы на стороне публикации и на стороне подписки должны быть обычными, а не представлениями, мат. представлениями
- при репликации секционированных таблиц есть ограничения до 13 версии - вы можете реплицировать секции одну в одну, но нельзя реплицировать данные в таблицы, секционированные по-другому
- *данные последовательностей не реплицируются*

https://aristov.tech

https://aristov.tech

Логическая репликация. Сервер подписки

- получают и применяют изменения
- может быть сколько угодно, зависит только от количества слотов репликации
- применение изменений происходит без выполнения команд SQL и связанных с этим накладных расходов на разбор и планирование, что уменьшает нагрузку на подписчика
- возможны конфликты с локальными данными
- *триггеры срабатывают для каждого подписчика отдельно*

https://aristov.tech

https://aristov.tech

Логическая репликация. Особенности

Варианты идентификации строки для изменения и удаления:

- столбцы первичного ключа (по умолчанию)
- столбцы указанного уникального индекса с ограничением NOT NULL
- все столбцы
- без идентификации (для системного каталога)

При конфликтах (нарушение ограничений целостности):

- репликация приостанавливается до устранения конфликта вручную
- при этом об этом вы никак не узнаете - нужно мониторить слот репликации!!!

Варианты исправить ситуацию:

- либо исправить данных
- либо пропустить конфликтующую транзакцию

https://aristov.tech

https://aristov.tech

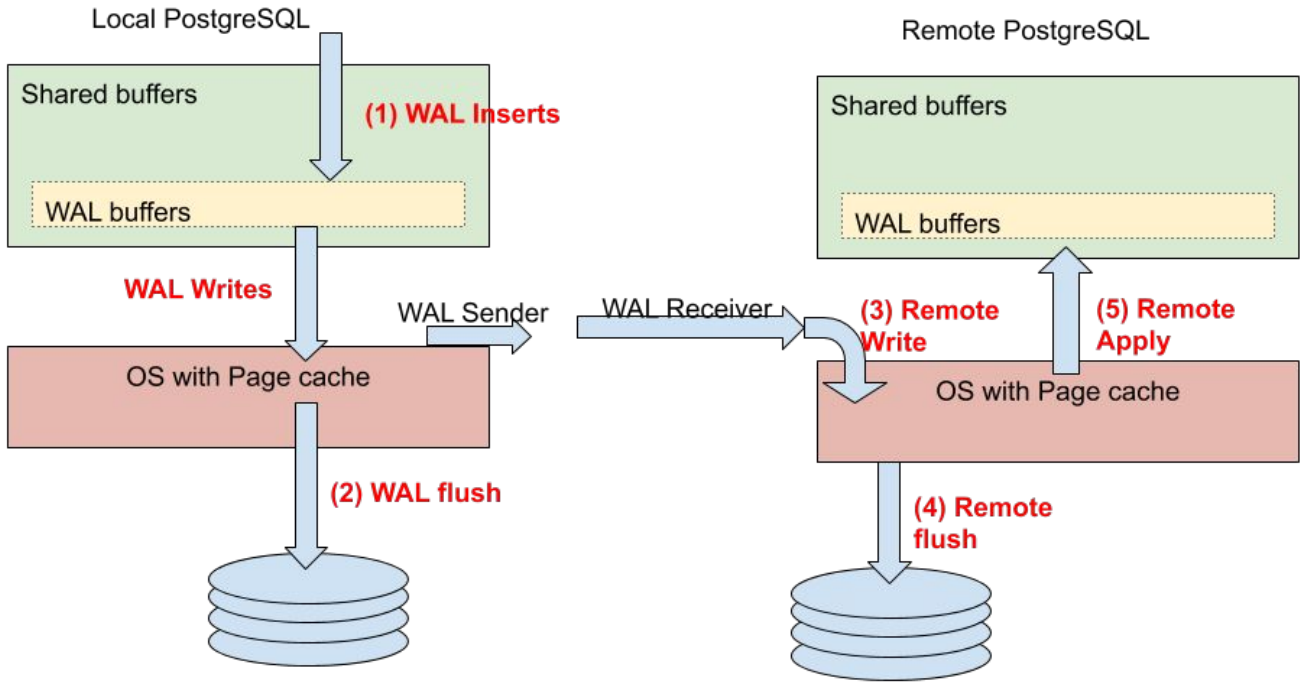
Кризис трех лет

https://aristov.tech

https://aristov.tech

Физическая репликация

Почему реплики узнают новости позже родителей:



https://www.percona.com/blog/postgresql-synchronous_commit-options-and-synchronous-standby-replication/

https://aristov.tech

https://aristov.tech

Физическая репликация

Особенности синхронной репликации (обязательно стендбай неймс - на практике посмотрим):

synchronous_commit Modes				
synchronous_commit setting	local durable commit	standby durable commit after PG crash	standby durable commit after OS crash	standby query consistency
remote_apply	•	•	•	•
on	•	•	•	
remote_write	•	•		
local	•			
off				

https://postgresqlco.nf/doc/en/param/synchronous_commit/

https://aristov.tech

https://aristov.tech

Физическая репликация

Уровень репликации:

At each transaction level

In a perfectly tunned application design, specific transaction can opt-in for specific sychronous_commit levels for each transaction, for example:

SET LOCAL synchronous_commit = 'remote_write';

At session level

The setting can be specified at each session-level so that it is applicable across the session unless overridden by the transaction level setting mentioned above.

SET synchronous_commit = 'remote_write';

https://aristov.tech

https://aristov.tech

Физическая репликация

At user level

In an ideal system, with well-managed user accounts, each user account will be dealing with specific functionality. It can range from the critical transaction system to the reporting user accounts. For example:

Shell

ALTER USER trans_user SET synchronous_commit= ON;

At database level

Specifying at the database level is useful when we have dedicated systems for reporting or temporary staging information.

ALTER DATABASE reporting SET synchronous_commit=OFF;

At instance level

This is at the PostgreSQL instance level as follows:

ALTER SYSTEM SET synchronous_commit=OFF;

https://aristov.tech

https://aristov.tech

Пошла теплая (реплика)

Как снизить нагрузку?

Правильно, ребята!

Брать WAL не с мастера, а из бэкапов!

Для этого необходимо настроить:

- hot_standby = off - не принимаем сетевые подключения (классика)
- archive_mode = on
- archive_command
- restore_command
- пустой файл standby.signal в каталоге данных реплики обязателен

Лучше через любую из утилит бэкапов с прошлого ОУ и включить сжатие

https://aristov.tech/blog/otkrytye-lekczii-ot-aristov-tech/#backups

https://aristov.tech

https://aristov.tech

Каскадное воспитание

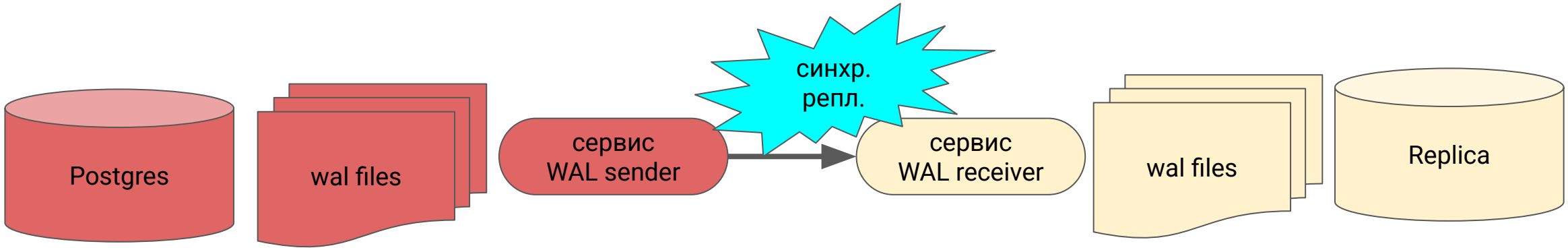
https://aristov.tech

https://aristov.tech

Горячий резерв for high availability

Синхронная репликация:

- реплика должна максимально соответствовать мастеру
- запросы к реплике возможны, но не приоритетны
- количество и имена в параметре synchronous_standby_names = 'FIRST/ANY 2 (s1, s2, s3)'
- synchronous_standby_names = '*'



Параметры primary:

synchronous_commit = on

Параметры replica:

hot_standby_feedback = off - не сообщаем о транзакциях мастеру

max_standby_streaming_delay=t - маленькое время отмены конфл.транз.

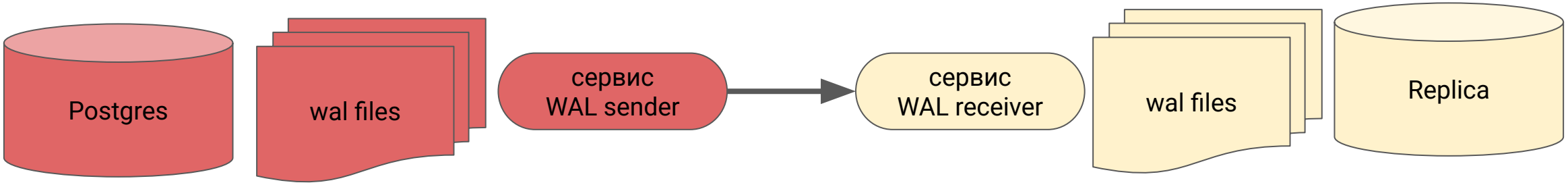
https://aristov.tech

https://aristov.tech

Балансировка OLTP

Много коротких запросов:

- запросы на реплике должны отрабатываться
- долгие запросы повлияют на primary, но у нас их не должно быть



Параметры primary:

synchronous_commit = off/local

Параметры replica:

hot_standby_feedback = on - сообщаем о транзакциях мастеру

max_standby_streaming_delay=T - большое время отмены конфл.транз.

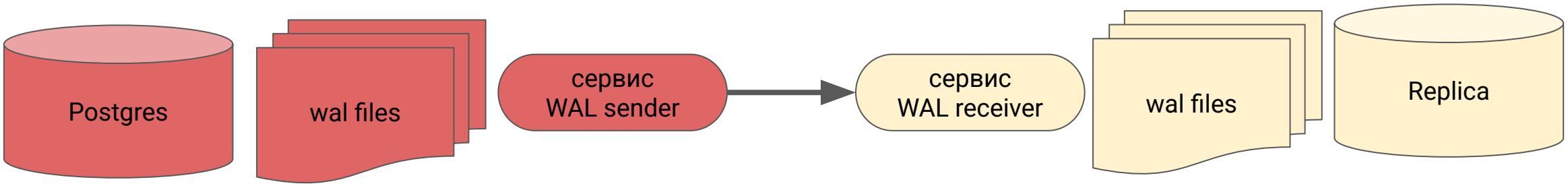
https://aristov.tech

https://aristov.tech

Реплика для отчетов

Аналитическая реплика:

- запросы на реплике должны отрабатывать, в том числе и долгие. Самые актуальные данные не нужны
- мастер не должен испытывать от этого неудобства



Параметры primary:

synchronous_commit = off

Параметры replica:

hot_standby_feedback = off - не сообщаем о транзакциях мастеру

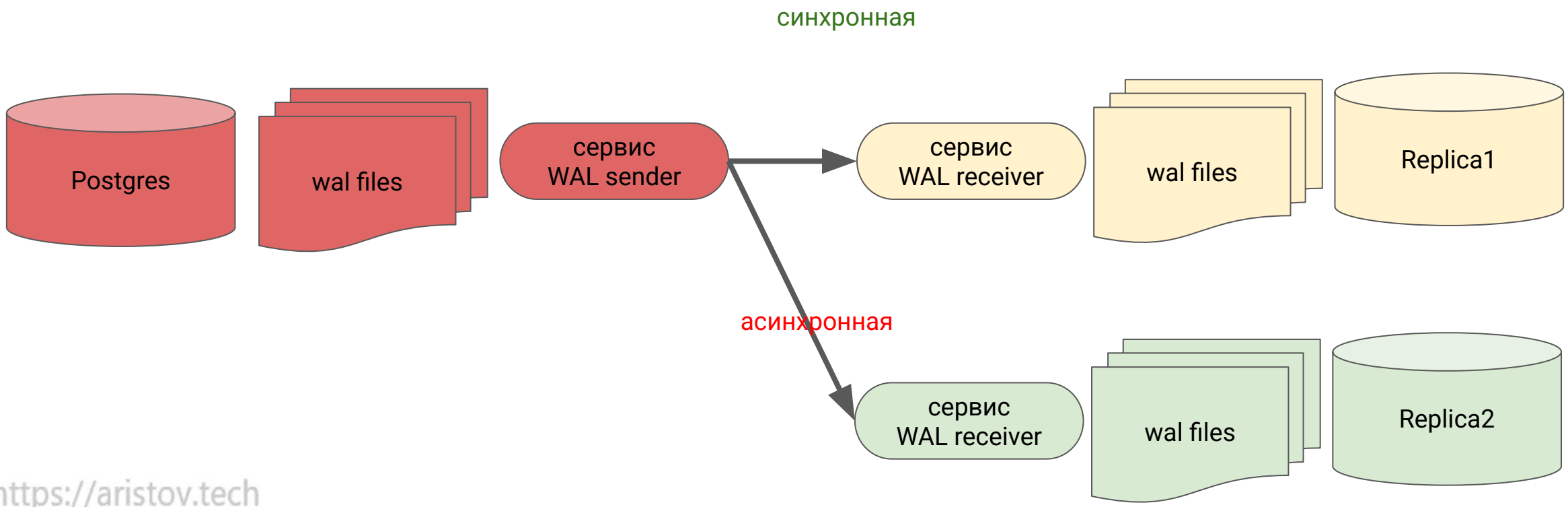
max_standby_streaming_delay=-1 - отключаем отмену конфл.транз.

https://aristov.tech

https://aristov.tech

Горизонтальное масштабирование с HA

- Синхронная реплика для отказоустойчивости
- Асинхронная для OLTP нагрузки



https://aristov.tech

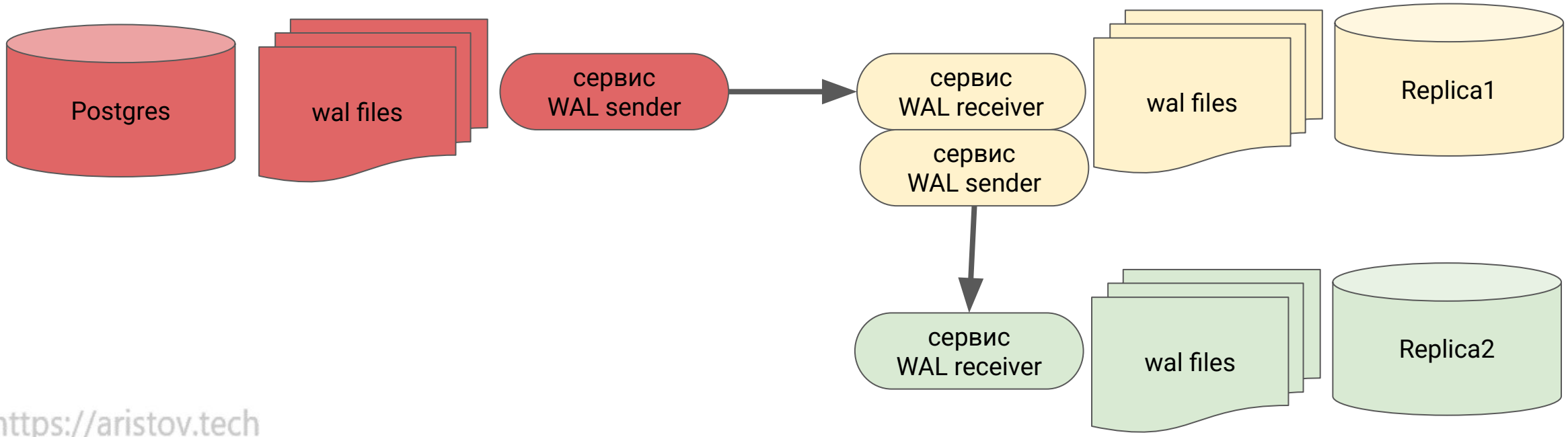
https://aristov.tech

Если реплик стало много и старшие следят за младшими

Каскадная репликация.

Плюсы и минусы «дедовщины» в кластере:

- Для снижения сетевой нагрузки и нагрузки на жесткий диск primary сервера
- мы не узнаем, что мастер умер



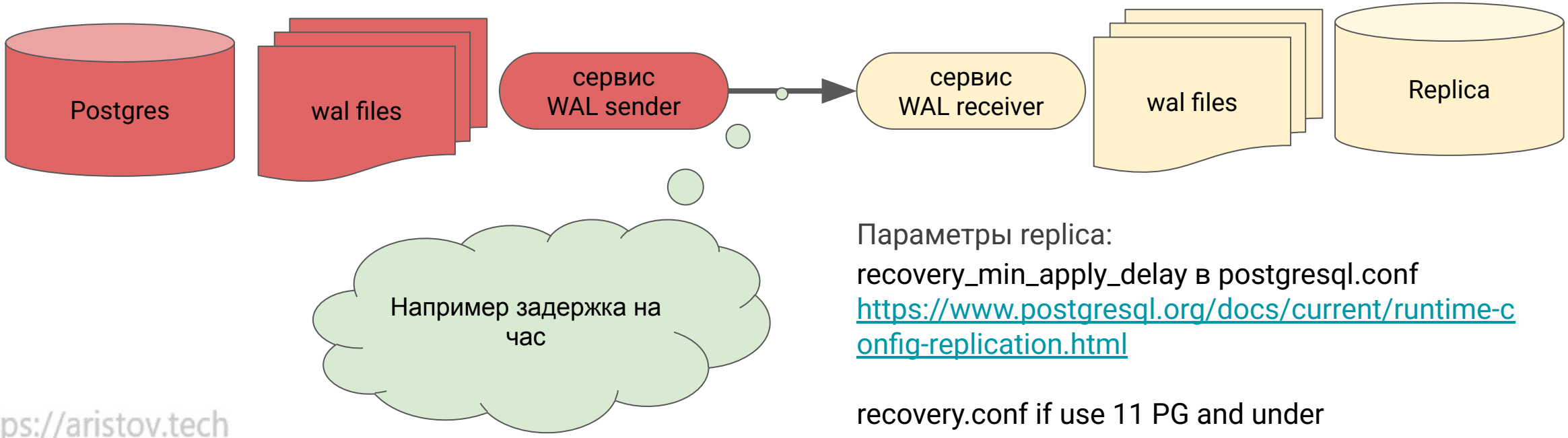
https://aristov.tech

https://aristov.tech

Time machine

Изменения на реплике применяются с задержкой. Например, зачем это может понадобиться:

- произошел фатальный сбой по той или иной причине и мы можем остановить репликацию до битой транзакции
- можно поставить репликацию на паузу для проведения опасных миграций



Параметры replica:

recovery_min_apply_delay в postgresql.conf
https://www.postgresql.org/docs/current/runtime-config-replication.html

recovery.conf if use 11 PG and under

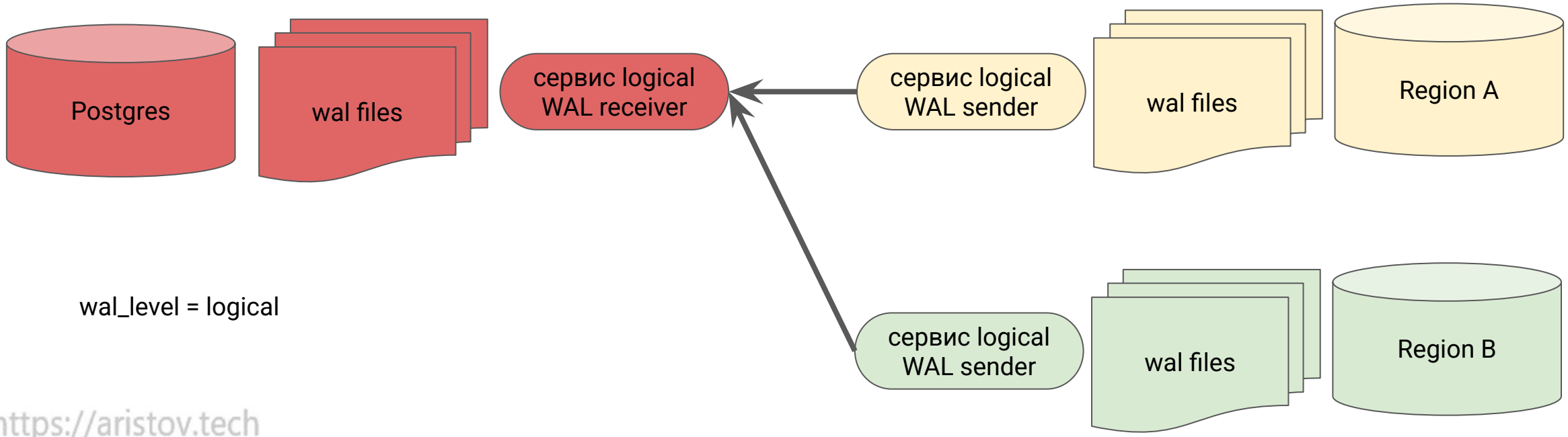
!!!synchronous_commit is set to remote_apply+ wait commit!!!

https://aristov.tech

https://aristov.tech

Объединение данных

- Логическая репликация
- Можем выбирать отдельные таблицы для репликации
- Публикация+подписка
- Не забываем про уникальность ПК

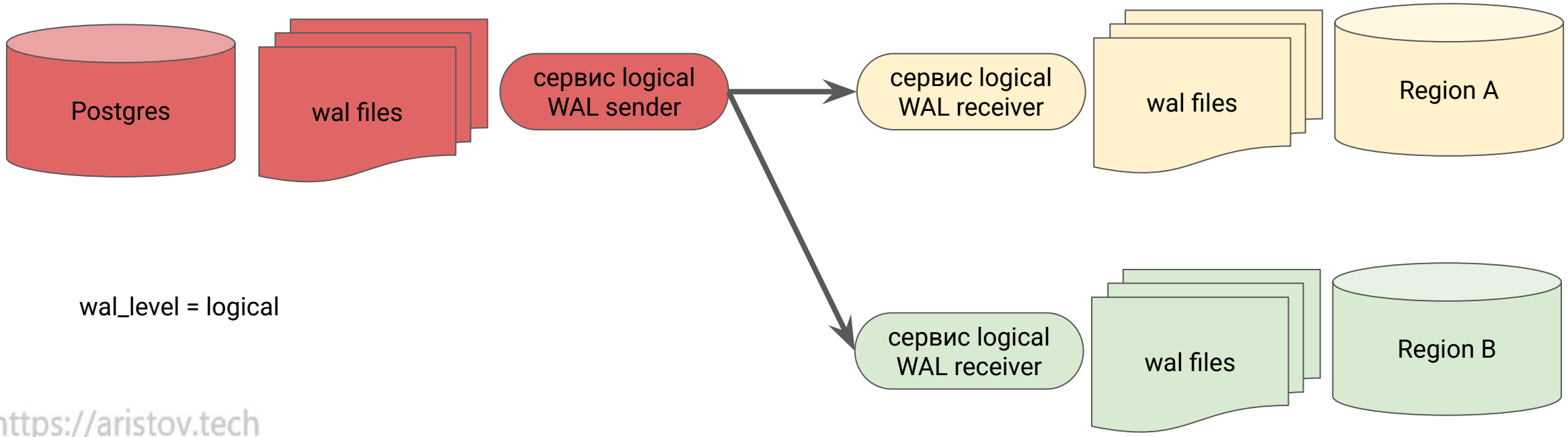


https://aristov.tech

https://aristov.tech

Рассылка данных

- Логическая репликация
- Можем выбирать отдельные таблицы для репликации
- можем комбинировать с предыдущим вариантом

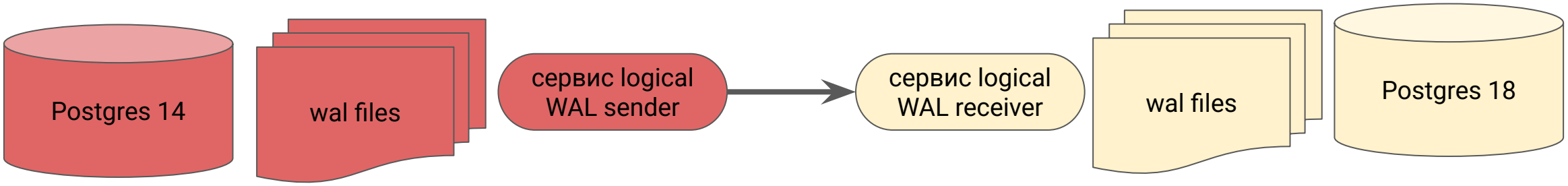


https://aristov.tech

https://aristov.tech

Смена версии ПГ

- Единственный вариант онлайн поменять версию ПГ



wal_level = logical

https://aristov.tech

https://aristov.tech

«Детские пособия», которые разоряют семью

https://aristov.tech

https://aristov.tech

Что гарантируют слоты репликации

- Master отслеживает позицию реплики (LSN — Log Sequence Number) через активный слот. Он **физически не удалит ни один WAL-файл** из своего каталога pg_wal, пока **реплика не подтвердит**, что прочитала и применила его
- Если вы используете Hot Standby (чтение на реплике) и включаете параметр vacuum_defer_cleanup_age или обратную связь hot_standby_feedback, слот гарантирует, что процессы очистки (VACUUM) на Master не удалят мертвые строки в таблицах, если эти строки все еще читаются открытыми транзакциями на реплике
- Если используется логическая репликация, слот дополнительно гарантирует, что Master сохраняет не только WAL, но и внутреннюю структуру каталогов (системный каталог), необходимую для декодирования данных в SQL-команды начиная с точной позиции реплики.если реплика упадет надолго или сломается навсегда, Master продолжит бесконечно копить WAL-файлы

https://aristov.tech

https://aristov.tech

Реплика «ушла в армию»

- Если реплика упадет надолго или сломается навсегда, при включенном слоте репликации Master продолжит бесконечно копить WAL-файлы, что приведёт к окончанию места на диске и аварийной остановке Постгреса
- Если слот репликации отключён, реплика по возвращении в строй может не найти нужного WAL и придётся её переналивать

https://aristov.tech

https://aristov.tech

Когда они вырастают

https://aristov.tech

https://aristov.tech

Папа «прилег отдохнуть»

По умолчанию в Postgres встроен только ручной механизм переключения, а для автоматического требуются внешние инструменты.

Реплики продолжают работать на чтение.

- Асинхронная репликация (synchronous_commit = off или local): Мастер подтверждает клиенту фиксацию транзакции (COMMIT), как только она записалась в его локальный WAL-лог. Если мастер «умирает» до того, как процесс walsender передал эти данные на реплику, возникает потеря данных (Data Loss). После продвижения реплики эти транзакции будут потеряны.
- Синхронная репликация (synchronous_commit = on или remote_write): Мастер не вернет клиенту успешный ответ, пока WAL не запишется на реплику (или в ее память при remote_write). Данные в безопасности. Но если реплика тоже упадет, мастер заблокирует любые новые записи.

https://aristov.tech

https://aristov.tech

Patroni - нотариус для делёжки наследства

По умолчанию в Postgres встроен только ручной механизм переключения, а для автоматического требуются внешние инструменты.

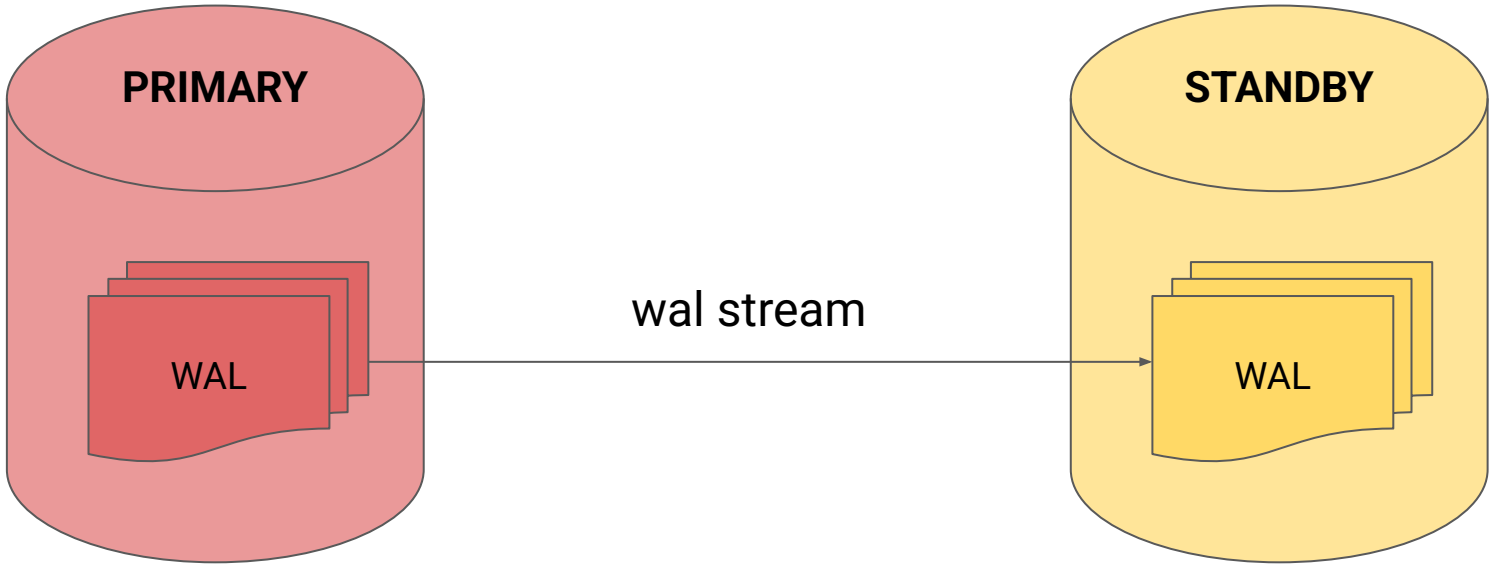
Реплики продолжают работать на чтение.

- Асинхронная репликация (synchronous_commit = off или local): Мастер подтверждает клиенту фиксацию транзакции (COMMIT), как только она записалась в его локальный WAL-лог. Если мастер «умирает» до того, как процесс walsender передал эти данные на реплику, возникает потеря данных (Data Loss). После продвижения реплики эти транзакции будут потеряны.
- Синхронная репликация (synchronous_commit = on или remote_write): Мастер не вернет клиенту успешный ответ, пока WAL не запишется на реплику (или в ее память при remote_write). Данные в безопасности. Но если реплика тоже упадет, мастер заблокирует любые новые записи.

https://aristov.tech

https://aristov.tech

Patroni - нотариус для делёжки наследства

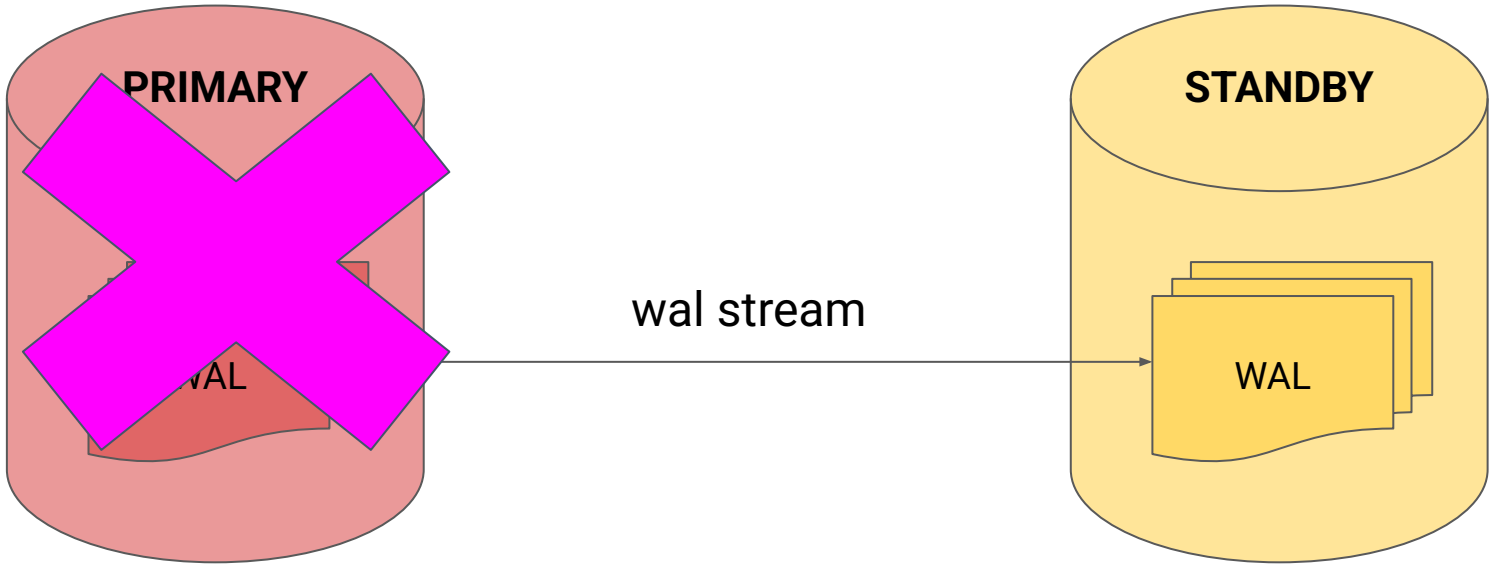


2 ноды. Всё настроили и допустим умер мастер. Что произойдёт?

https://aristov.tech

https://aristov.tech

Кластер из 2 нод



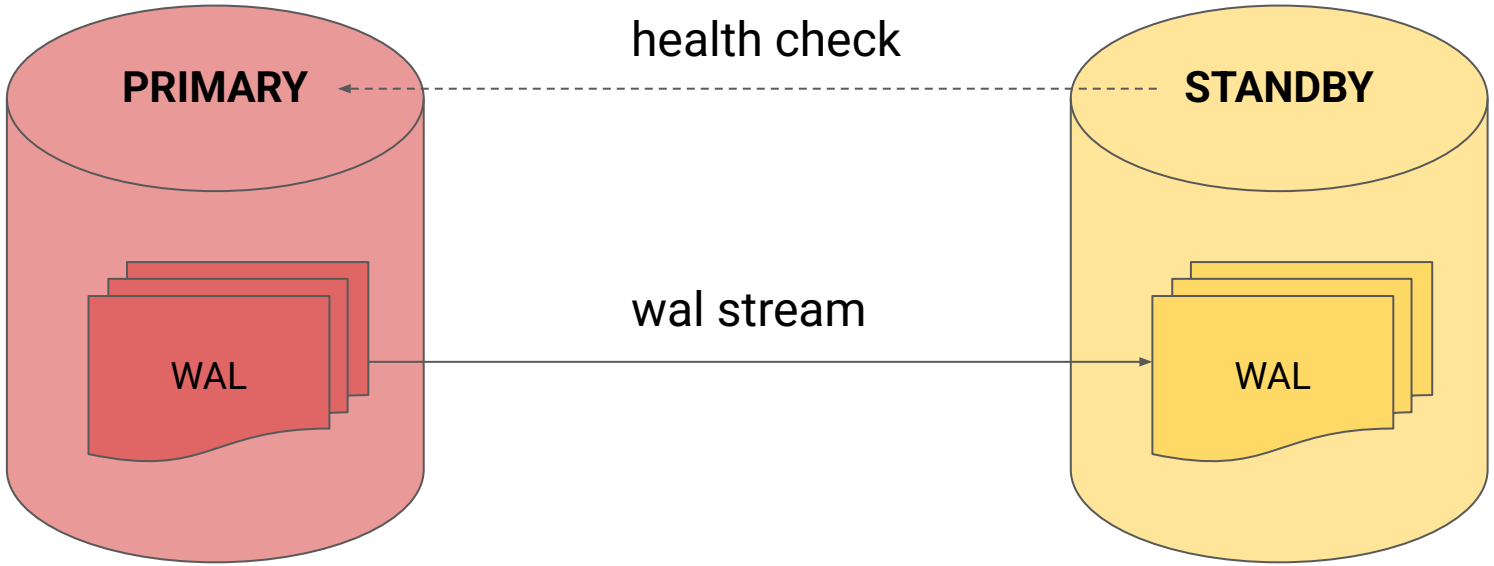
Всё настроили и допустим умер мастер. Что произойдёт?

Правильный ответ - ну вот и всё) Кто будет standby промоутить?

https://aristov.tech

https://aristov.tech

Кластер из 2 нод

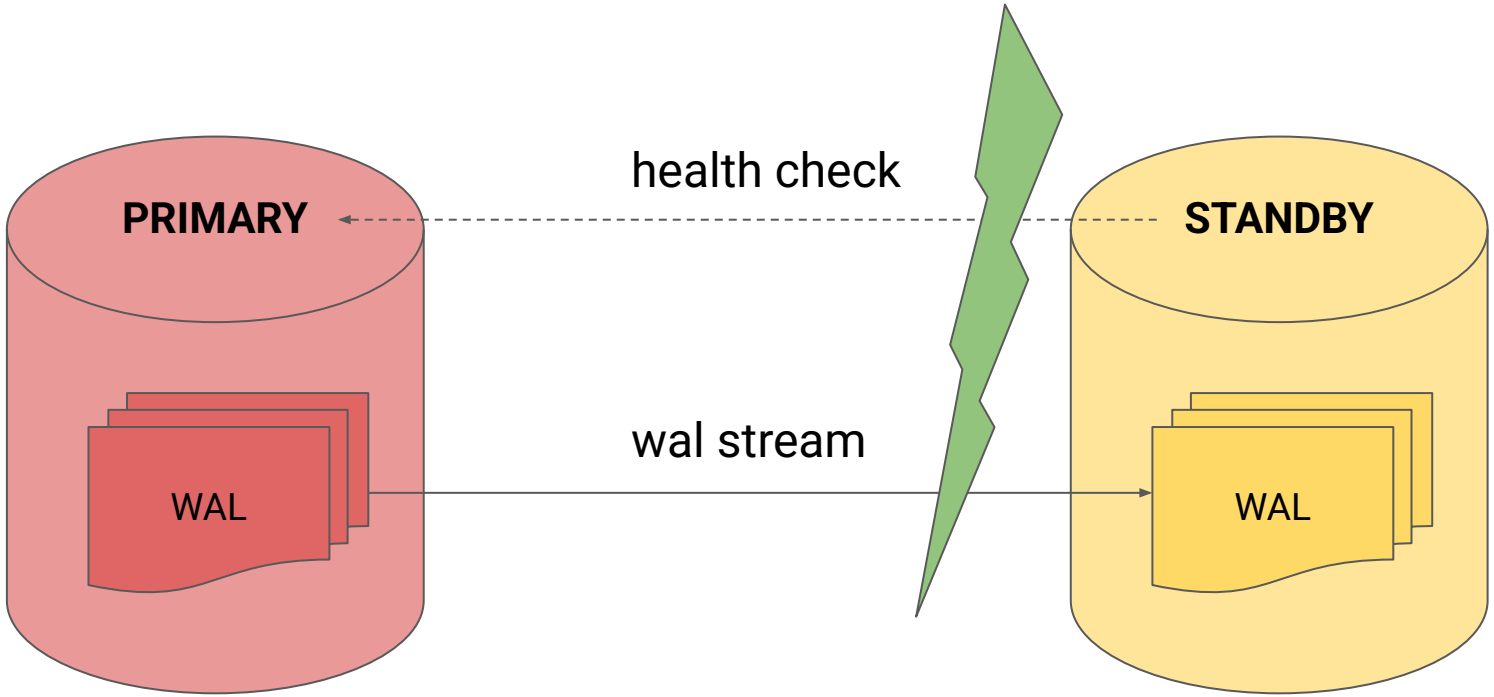


Добавили хелсчек. Ну теперь то всё хорошо?

https://aristov.tech

https://aristov.tech

Кластер из 2 нод



Допустим произошел обрыв связи, что произойдет?

https://aristov.tech

https://aristov.tech

Кластер из 2 нод



Допустим произошел обрыв связи, что произойдет?

Сплитбрейн !!!

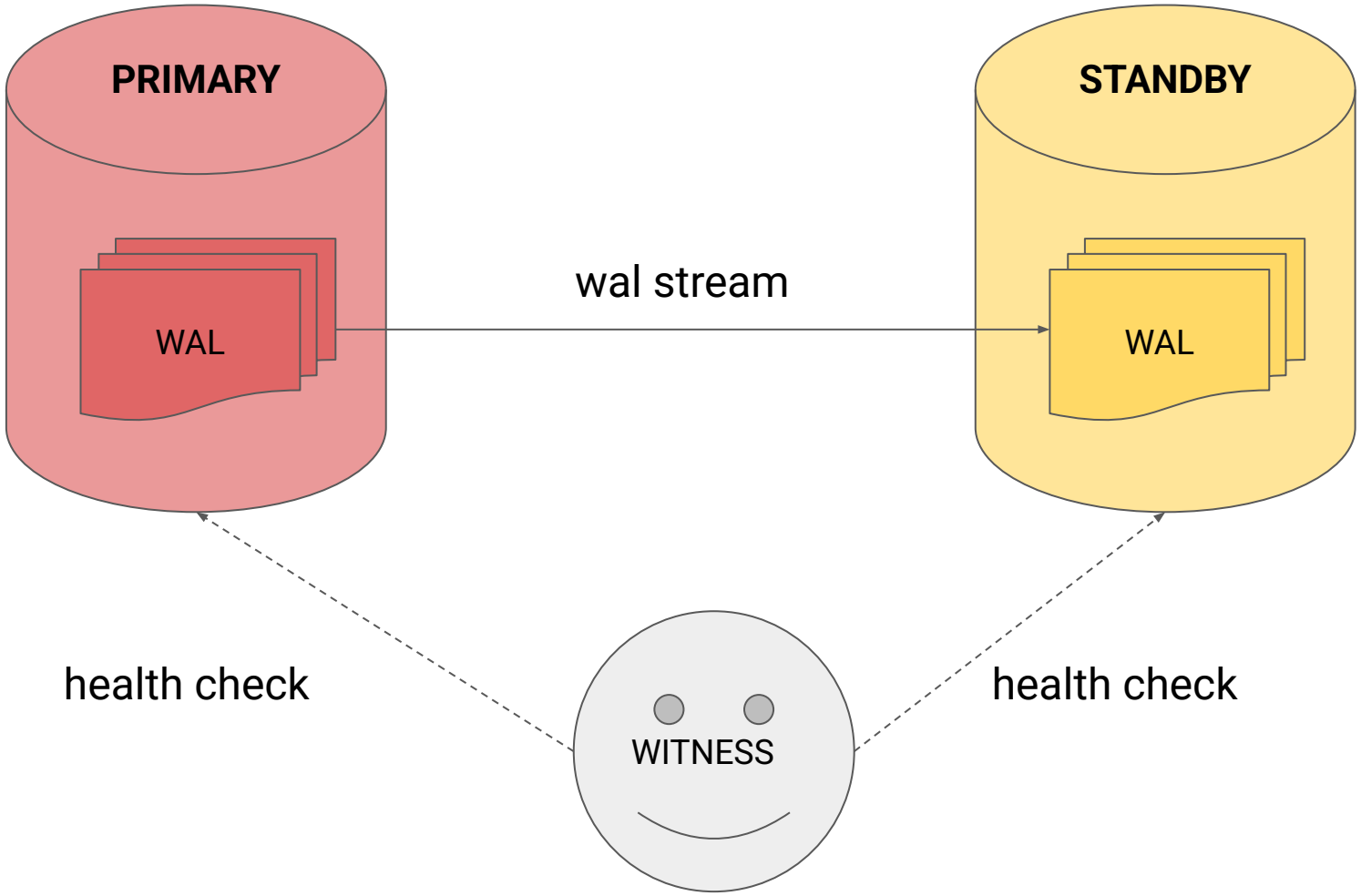
https://en.wikipedia.org/wiki/Split-brain_(computing)

https://aristov.tech

https://aristov.tech

Кластер из 2 нод with Witness

Что может пойти не так??

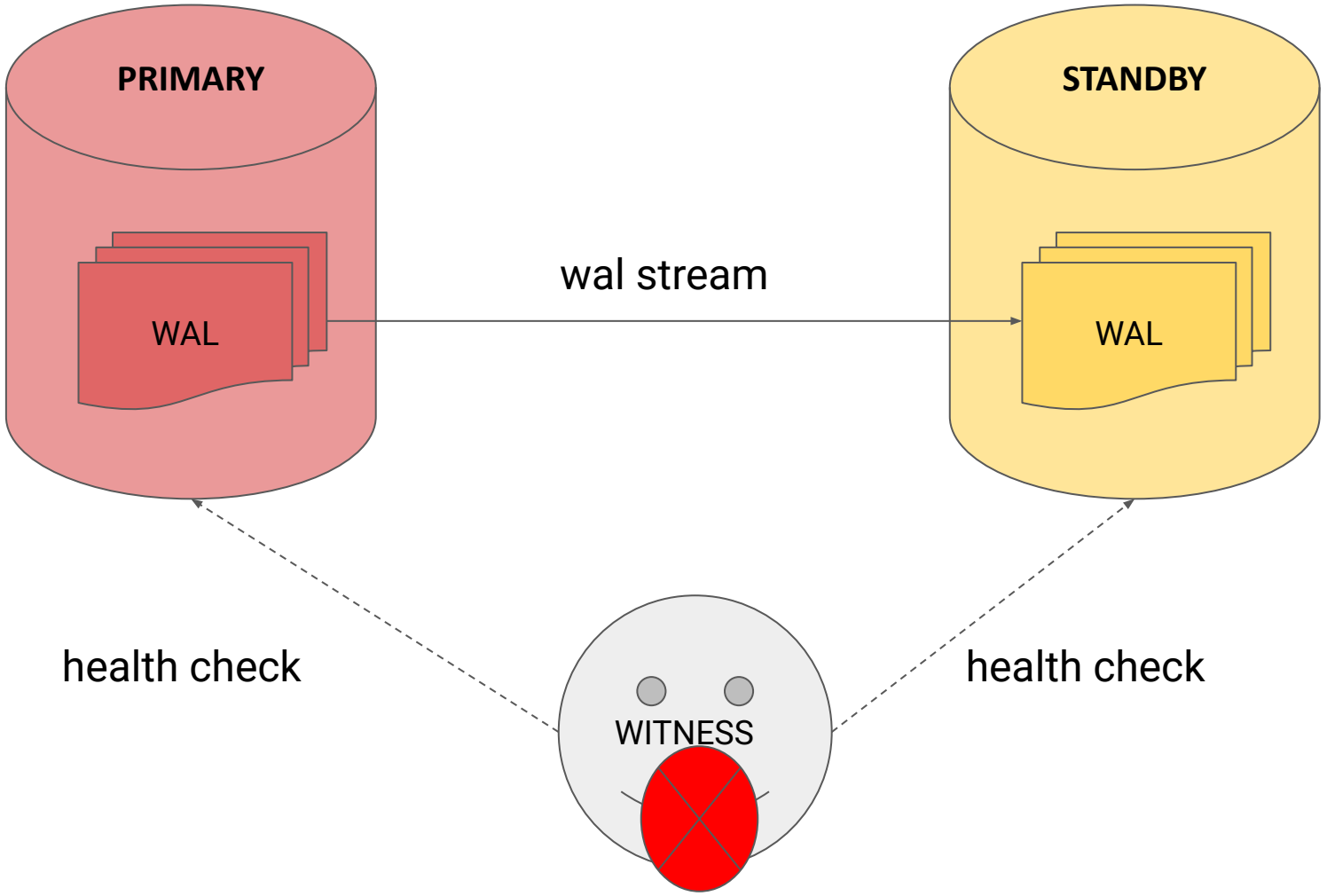


https://aristov.tech

https://aristov.tech

Кластер из 2 нод with Witness

Умер наблюдатель

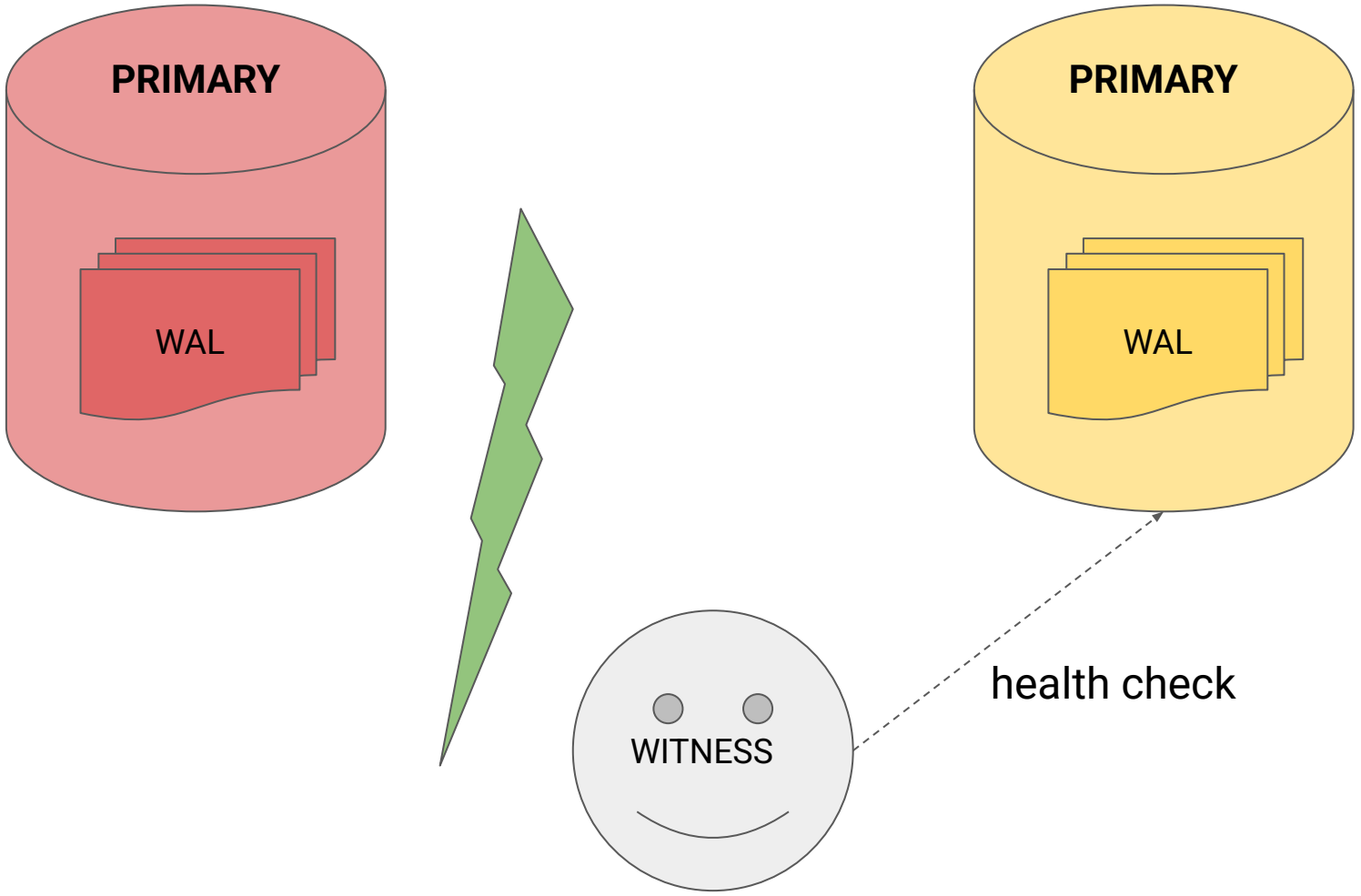


https://aristov.tech

https://aristov.tech

Кластер из 2 нод with Witness

Обрыв соединения с основной нодой. Казалось бы и что? А снова сплитбрейн(

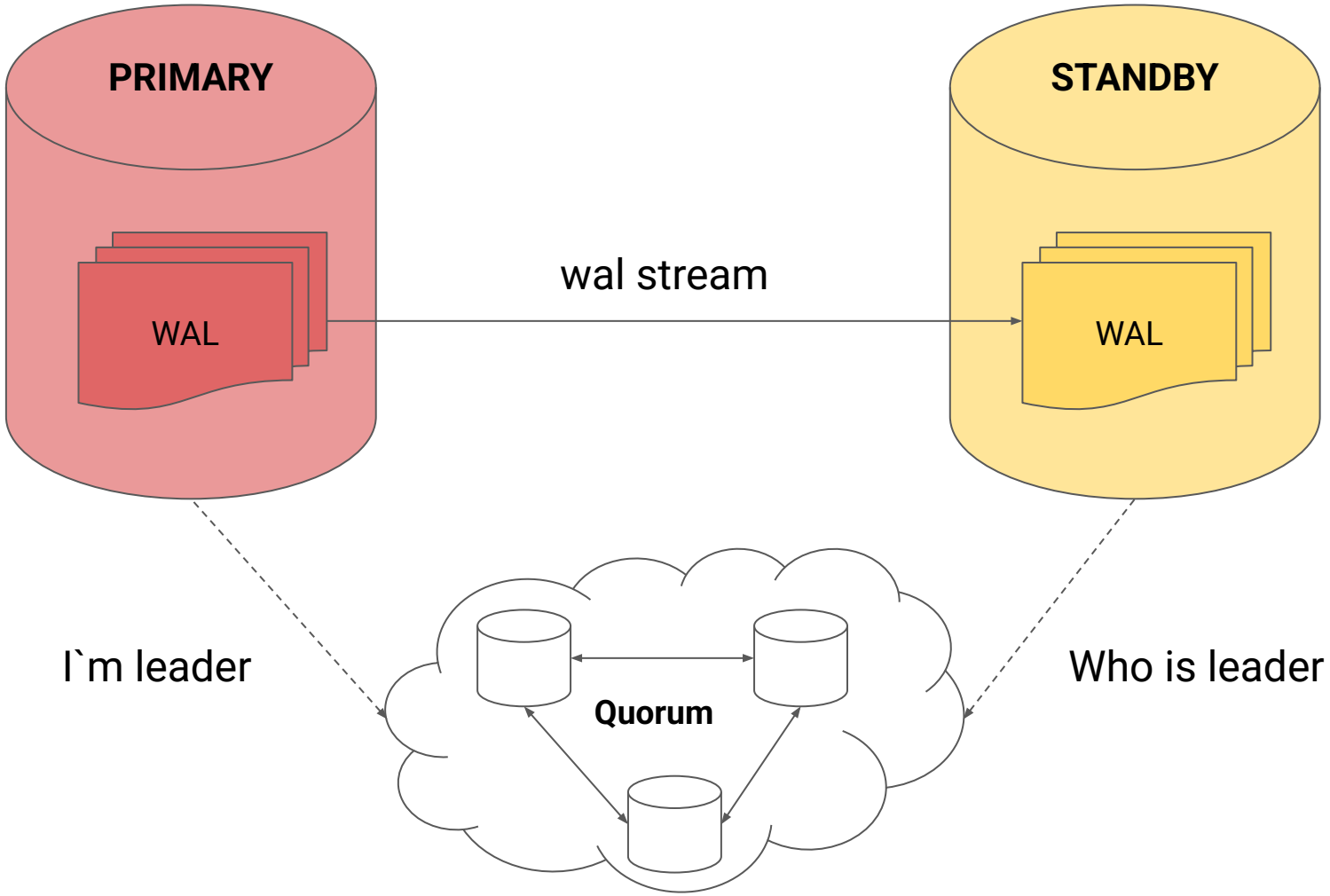


https://aristov.tech

https://aristov.tech

Кластер из 2 нод с ETCD/CONSUL/ZOOKEEPER

Одно из решений (Patroni)



https://aristov.tech

https://aristov.tech

Практика

https://aristov.tech

https://aristov.tech

Проект aristov.tech

https://aristov.tech

https://aristov.tech

aristov.tech

Книги по 16-18 Постгресу (Оптимизации/Архитектуре) https://aristov.tech/#orderbook

По 14 версии доступна бесплатно!!! https://aristov.tech/Postgres14book.pdf

Эксклюзивный **курс** по Оптимизации Постгреса - 10 поток - постоянно дорабатывается исходя из бизнес-кейсов https://aristov.tech/blog/kurs-po-optimizaczii-postgresql/ - **старт 30 января**
Со всеми **отзывами** без цензуры можно ознакомится по ссылке
https://aristov.tech/blog/otzivi-kurs/

Блог с популярными темами https://aristov.tech/blog
ТГ **канал** с новостями блога и проектаhttps://t.me/aristov_tech
Ютуб канал с интересными видео https://www.youtube.com/@aristovtech
Рутуб - https://rutube.ru/u/aristovtech/
ВКВидео - https://vkvideo.ru/@aristov.tech
Моя **группа** ДБА https://t.me/dbaristov - уже ~400 экспертов

https://aristov.tech

https://aristov.tech

aristov.tech

Открытие вебинары (9 штук + этот) - https://aristov.tech/blog/otkrytye-lekczii-ot-aristov-tech/
Воркшопы (11) - https://aristov.tech/blog/vorkshopy-ot-aristov-tech/
Практические воркшопы Патрони по 4-5 часов (5) - https://aristov.tech/blog/patroni-on-premise/

Курс Patroni в записи (5 крутых лекций) - https://aristov.tech/blog/kurs-patroni/

7 B2B курсов https://aristov.tech/blog/linejka-kursov-dlya-b2b/

Также за прошедшее время был запущен проект **менторинга** в котором уже участвует 17 лучших экспертов в отрасли ДБА/DevOps и разработки.
Ознакомится с преподавателями и проектом https://aristov.tech/mentorship

Всегда можно со мной связаться через сайт для консультация, аудита вашего проекта, менторинга, обучения и многого другого

https://aristov.tech

https://aristov.tech

aristov.tech

Розыгрыш книг и скидки 30% на курс (только очные варианты)

регистрируемся по форме:

https://forms.gle/9i8eRcSHGLnMXi3B7

Сам розыгрыш в db-fiddle

https://www.db-fiddle.com/f/43wbuUzv7YUAcS3aypkvvC/1

https://aristov.tech

https://aristov.tech

ДЗ

https://aristov.tech

https://aristov.tech

ДЗ

1. Подписаться на канал
2. Подписаться на Ютуб
3. Вступить в мою группу ДБА
4. Посещать открытые уроки
5. Посещать воркшопы или посмотреть их в записи
6. Прийти на курс по оптимизации%)
7. Прокачаться с помощью менторинга
8. Устроиться на одну из вакансий 500+

П.С. Книги просто бомбические

https://aristov.tech

https://aristov.tech

Спасибо за внимание!

Увидимся, друзья

Аристов Евгений

https://aristov.tech

Все материалы защищены авторским правом. Использование полностью или частично разрешено только с письменного разрешения Аристов Е.Н. (с)